3C · AI ENGINEERING ALIGNMENT

A day in the life of a developer using 3C.

One engineer's working day inside a team that has adopted **3C — Claude Code Coach** at **L2**. The slash commands she runs, the gates she passes, the boundaries her team declared, and the things she never types by hand.

Standards

`CLAUDE.md` under a 150-instruction cognitive budget, `.claude/rules/`, the architecture pattern library, the security policy templates — the shared spine every AI session reads from.

Workflow

The `/3c-*` slash commands — `create-issue`, `implement`, `review`, `test`, `onboard`, `retrospective`, and the orchestrators `project-manager`, `programme-manager` and `review-board` — plus the terminal commands `3c bug` and `3c bypass`. The same lifecycle on every laptop.

Quality

Pre-Commit Hook (<10s) → Security Scan Gate (<60s), with an advisory post-edit check (~300ms) that blocks nothing. Every gate fired, every event logged.

Specialists on call: an eight-agent crew — `architect`, `code-reviewer`, `journey-architect`, `product-owner`, `qa-reviewer`, `security-reviewer`, `test-author` and `ux-expert` — dispatched when one perspective isn't enough; `/3c-review-board` runs four of them by default — plus `ux-expert` on UI diffs and `product-owner` when acceptance criteria are supplied — in parallel, and reconciles one verdict. **The why travels with the rule:** every team rule is plain markdown in `.claude/rules/`, versioned with the repo — the OWASP pack cites its source on the rule itself, and the team's own rules accumulate their *why* through the PRs that change them. **Honesty discipline:** one `git`

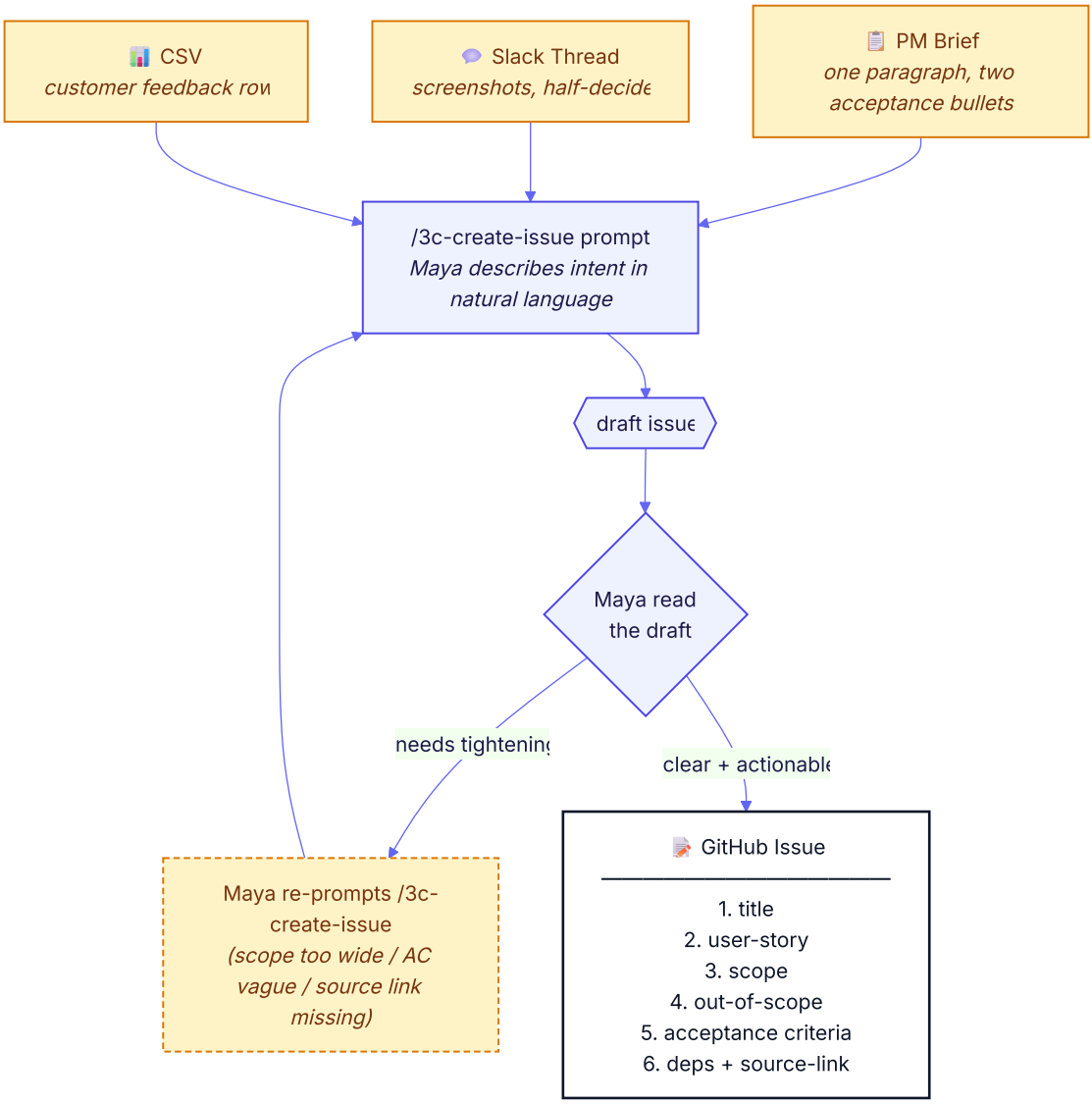
`revert` removes 3C entirely; no source code leaves the workstation; the framework under-claims by design.

POST-EDIT ADVISORY — WHAT MAYA SEES WHEN A WRITE CROSSES A LAYER

```
3c post-edit validation found 1 issue(s) — please self-correct:
  - [layer-boundary] src/account/home/orders-panel.tsx:3 —
    "app" layer must not import from "infra" layer
```

Raw inputs in. Reviewable issue out.

Maya's morning starts with a PM brief, a Slack thread, and a CSV of customer feedback. She invokes `/3c-create-issue`, iterates on the draft, and ends with one GitHub issue carrying six required fields. The assistant types the issue body; Maya picks intent and judges fit.



The exact prompt Maya sends, and the resulting issue body she reviews:

```
/3c-create-issue "account home: surface last 5 orders" --template=feature

We want logged-in users to see their last 5 orders on the
account home. PM brief attached (link).
Source: customer feedback CSV row 247, Slack #cx-feedback 2026-05-19.
Out of scope: changing order-detail page.
```

```
Title: account home: surface last 5 orders
User story: As a logged-in user, I want my last 5 orders…
Scope: account-home page, /api/orders pagination
```

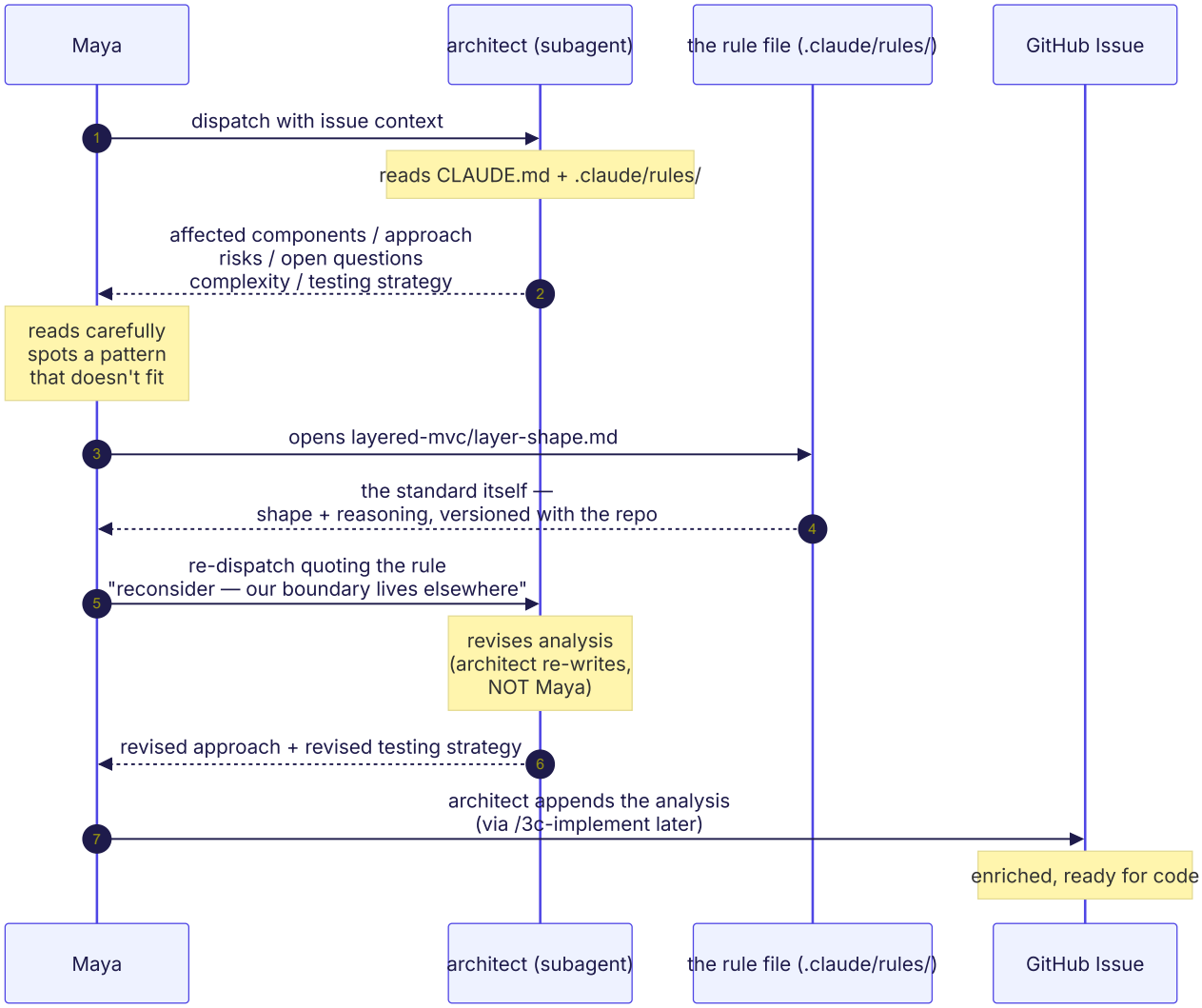
```
Out of scope: order-detail page, reordering, returns
AC1: orders panel shows 5 most-recent orders, descending by date
AC2: empty state when user has zero orders
AC3: link to full order history (existing route)
Deps: none new. Source: Slack #cx-feedback…
```

What 3C does for you here: turns three loose inputs into one structured artefact in under a minute. **What Maya still owns:** reading the draft and judging "is this what I meant?" — the only typing she does is the next prompt. **What 3C does NOT do:** read source files, scrape Slack, or transmit anything to GitHub except the title / body / labels / repo Maya explicitly provided.

> KEY TAKEAWAY
>
> The skill produces what you prompt, not what you mean. Iterating the prompt is the work; reading the draft is the gate.

Dispatch a specialist before code.

The issue is structured, not yet technical. Maya dispatches the `architect` subagent for an approach analysis, opens the rule file for a pattern she's unsure about, and re-prompts the architect with the rule's own reasoning until the analysis fits.



What the architect returns (first pass), and the rule file Maya quotes back — the directive her orders panel would cross:

```
Affected: src/account/home/, src/api/orders/list.ts
Approach: paginate via existing /api/orders?limit=5&sort=-date
Risks: stale-cache on order-creation (TTL 5 min on this route)
Open questions: empty state copy? confirm with PM
Complexity: S (≤ 1 day)
Testing: unit on pagination + integration on empty state
```

```
.claude/rules/patterns/layered-mvc/layer-shape.md

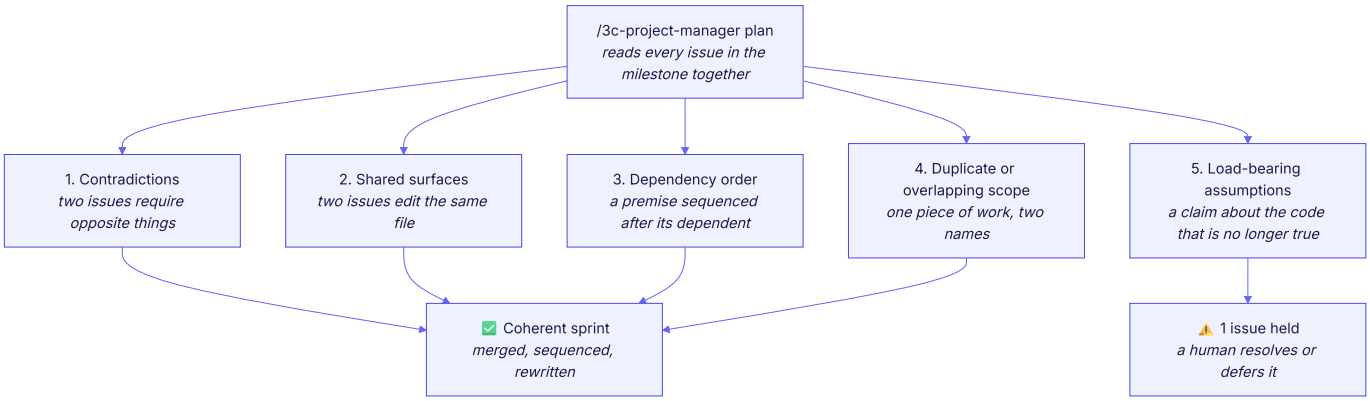
- Dependencies MUST flow downward only: Presentation MAY call Application;
  Application MAY call Persistence; the reverse MUST NOT hold.
```

What 3C does for you: dispatches a specialist that reads your standards before answering. **What Maya still owns:** noticing the analysis doesn't fit, and choosing to re-dispatch — the architect re-writes; Maya doesn't. **What 3C does NOT do:** design the solution unilaterally; the architect proposes, you accept or push back.

> KEY TAKEAWAY
>
> The architect is cheaper before code than after. One re-dispatch quoting the rule beats fifty minutes of refactoring.

Five checks, one coherent sprint.

With the sprint's candidate issues in a milestone, Maya runs `/3c-project-manager plan` before anyone writes code. It enriches each issue first — showing her a diff before it rewrites a body — and then its sprint-readiness gate reads every issue together: five checks for the inconsistencies a single-issue review cannot see, because each issue passes on its own while the set does not.



The invocation, and the coherence findings it reports (abridged — the skill reports them as a table):

```
/3c-project-manager plan "Sprint 24"
```

```
Sprint readiness — coherence, across all 12 issues:
- Contradictions: #1289 AC2 contradicts #1284 AC3 on empty-state. Decide.
- Shared surfaces: #1287 + #1291 both edit src/account/home/. Serialise.
- Dependency order: #1292 needs a util that #1295 refactors. Sequence.
- Duplicate or overlapping scope: #1288 and #1293 both add pagination. Merge.
- Load-bearing assumptions: #1290 assumes /api/orders sorts by date;
  it does not. Fix the issue body before it is built.
```

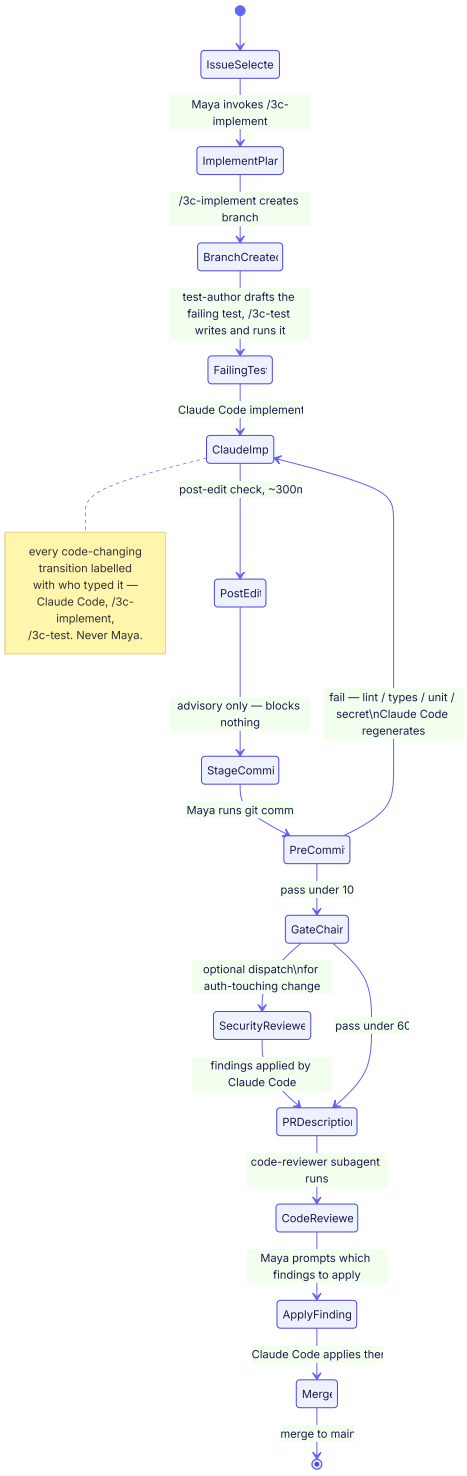
What 3C does for you: five cross-issue checks before the first branch exists, each finding naming the issues it concerns. **What Maya still owns:** choosing whether to merge, sequence, rewrite or hold each finding —

the skill reports and stops, she decides. **What 3C does NOT do:** reorder or rewrite the sprint for her; an unresolved finding stops the plan until a human resolves or defers it.

> KEY TAKEAWAY
>
> Catching a contradiction now costs five minutes. Catching it in PR review costs two days of rework.

One issue, every gate, zero hand-edits.

Maya picks issue #1284 and runs `/3c-implement 1284`. From branch creation through merge, every keystroke that changes code, tests, or the PR description is typed by Claude Code or by a `/3c-*` skill. Maya prompts, judges, and approves.



Claude Code reaches straight into the infra client from the orders panel. The post-edit hook checks that write against the layers her team declared in `.3c/architecture.yaml` and names the direction it crossed; the `arch-check` gate records the same finding into the run history her dashboard reads:

```
3c post-edit validation found 1 issue(s) — please self-correct:
  - [layer-boundary] src/account/home/orders-panel.tsx:3 —
```

```
    "app" layer must not import from "infra" layer
```

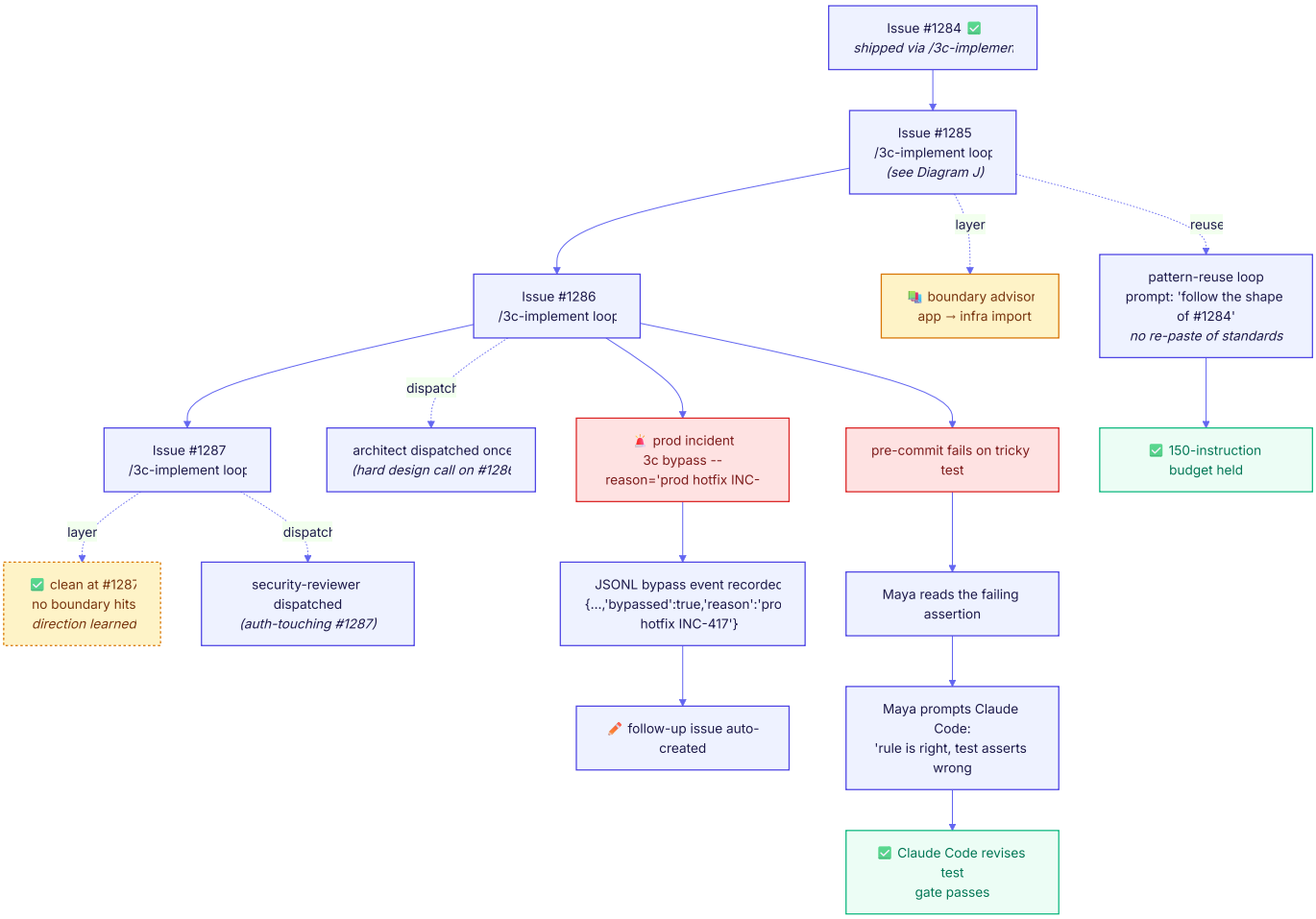

```
{"schema_version":1,"hook_id":"architecture-
boundary","trigger_event":"PreToolUse","summary":"arch-
check","exit_code":2,"failure_category":"validation","duration_ms":118,"network_used":false,"
05-20T14:32:11Z"}
```


What 3C does for you: the gates fire automatically; every finding names the rule, the file, and the line it fired on; every event is recorded. **What Maya still owns:** reading the advisory and deciding — "the boundary is right, route this through the domain layer" vs "the boundary is wrong, change the rule through a PR". **What 3C does NOT do:** fix the code for her unbidden — every revision happens because Maya re-prompted Claude Code with the diagnosis.

> KEY TAKEAWAY
>
> The gate chain is mechanical. The instruction is human. The keystroke is always the assistant's.

Compounding is the product.

After issue #1284 ships, Maya runs issues #1285, #1286, #1287 through the same loop. Boundary findings fall away as the declared architecture becomes second nature. New prompts reference shipped PRs as patterns, holding the 150-instruction budget. The hot-fix path and the tricky-failure path both route through the assistant — never around it.



The stanza of `.3c/architecture.yaml` the post-edit hook checked #1285 against, and the JSONL bypass event when she ships a hot-fix:

```
layers:
  app:    ["src/account/**"]
  domain: ["src/domain/**"]
  infra:  ["src/infra/**"]
deny:
  - ["app", "infra"]   # app must not import from infra
```

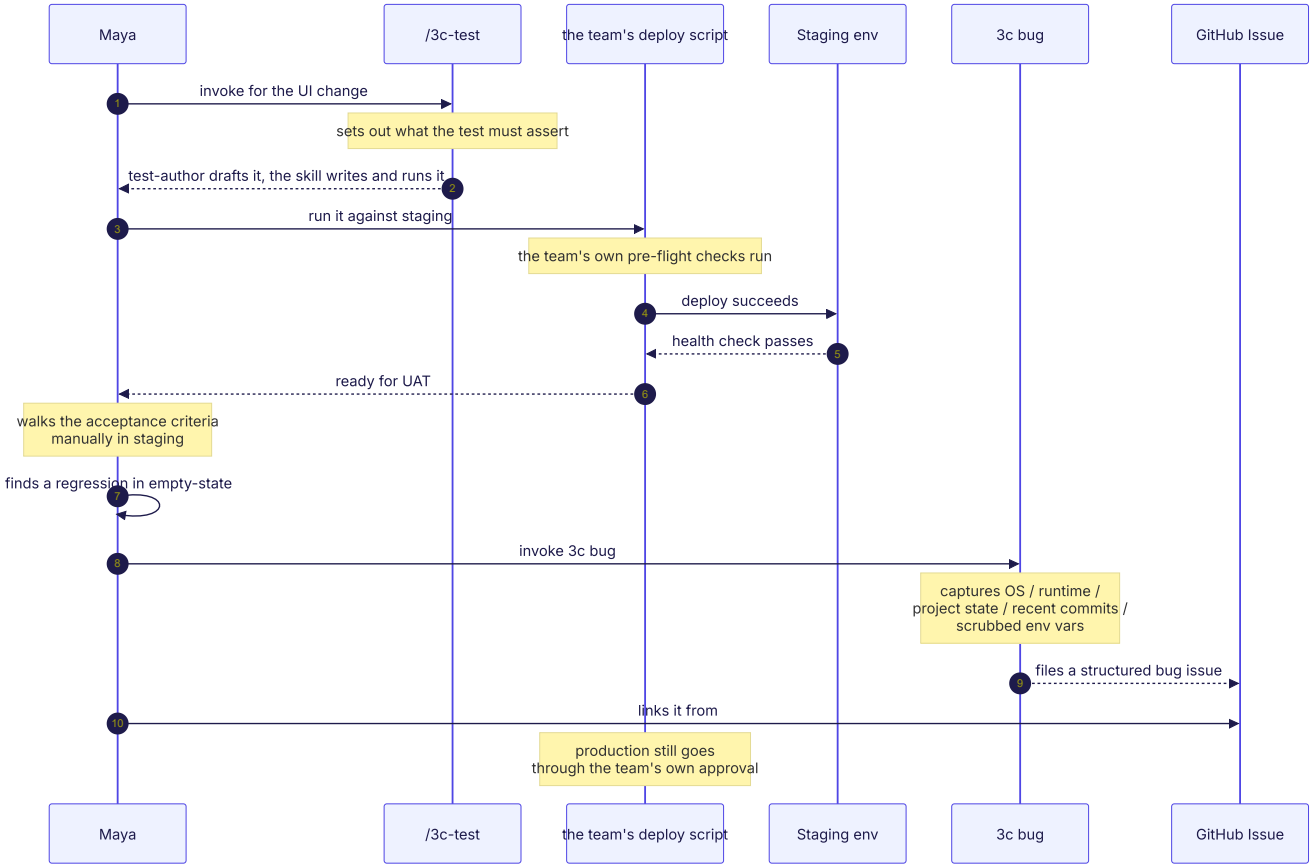
```
{"schema_version":1,"hook_id":"pre-commit","trigger_event":"PreToolUse","summary":"bypass:
prod hotfix #INC-
417","exit_code":0,"failure_category":"none","duration_ms":94,"network_used":false,"self_corr
05-20T16:14:02Z"}
```

What 3C does for you: the boundary check runs on every write, prompts get shorter, gates fire the same. Velocity compounds because the team's standards are doing the standardising, not the developers. **What Maya still owns:** the decision to bypass, the rationale she writes, the follow-up issue she files after shipping. **What 3C does NOT do:** erase the bypass — every event is permanent and reviewable.

> KEY TAKEAWAY
>
> The framework gets out of your way as you internalise it. By issue four, the boundary check has nothing left to say.

Walk the path. File the gap.

For #1287 (a UI change), `/3c-test` sets out what the integration test must assert; the `test-author` subagent drafts it, and the skill writes and runs it, test-first. Maya deploys to staging with the team's own deploy script, walks the acceptance criteria by hand, and uses `3c bug` to file the one regression she found as a structured issue, with the environment captured and secrets scrubbed.



The header of the issue `3c bug` files for the #1287 regression (env values scrubbed):

```
Bug report — #1287 follow-up
OS: macOS 14.5 (Darwin 23.5.0)
Runtime: Node 20.12.2, npm 10.5.0
Project state: branch=feat/issue-1287..., commit=a8c4f2e
Recent commits: 4 in last 24h
Env (scrubbed): NODE_ENV=staging, FEATURE_FLAGS=[**SCRUBBED**]
Repro: navigate /account, log out, log in as user with 0 orders
Expected: empty-state copy. Actual: orders panel renders [object Object].
```

What 3C does for you: the integration test is specified against the change's behaviour rather than from memory, the environment is captured, the bug report is structured — no "works on my machine" archaeology. **What Maya still owns:** walking the acceptance criteria with human eyes; only humans notice that empty-state looks broken. **What 3C does NOT do:** deploy to production for her — production still goes through the team's own approval.

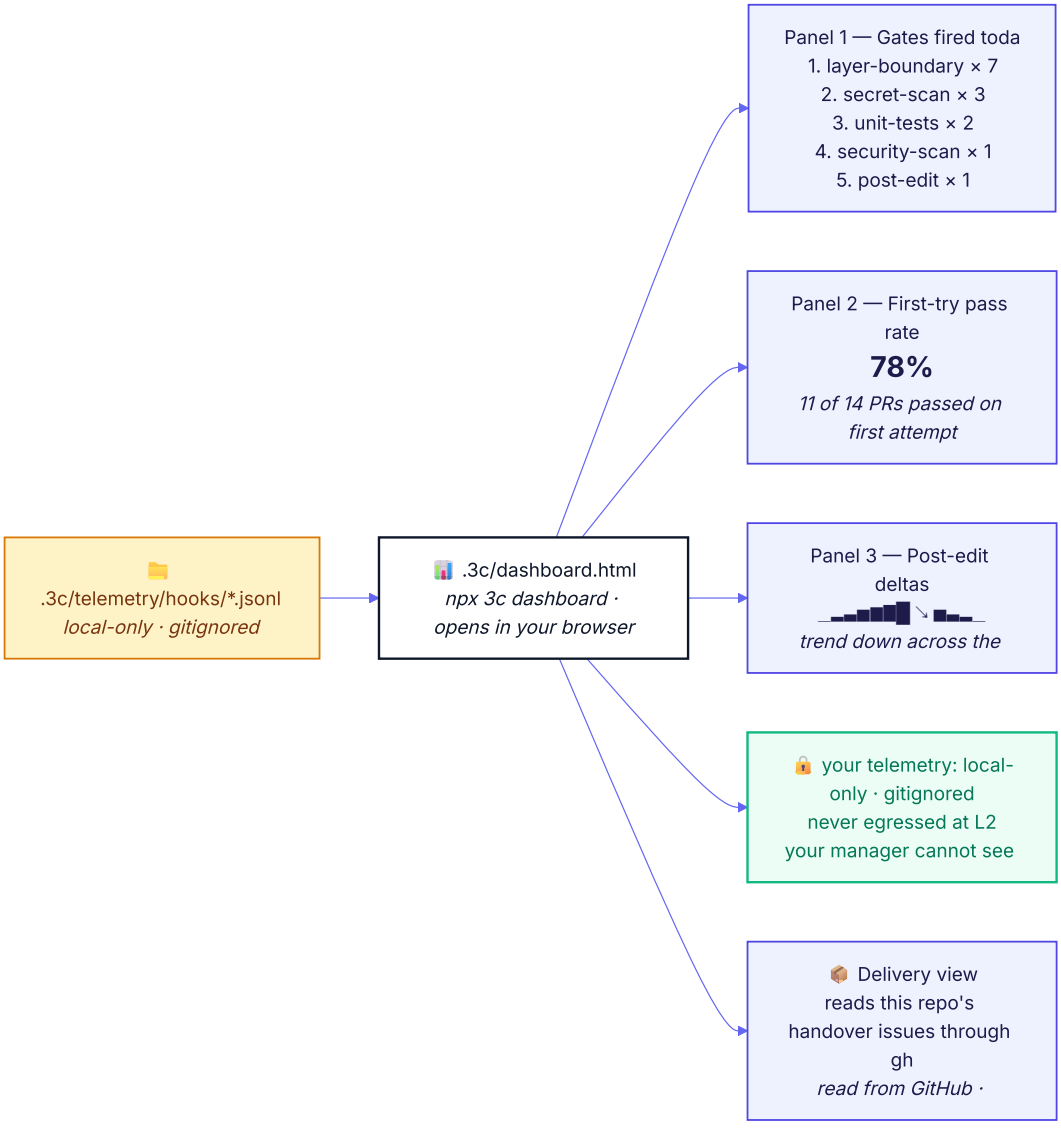
KEY TAKEAWAY

Manual UAT is the test the assistant cannot run. The framework's job is to capture what she found, structured.

END-OF-DAY

Your data. Your machine. Your reflection.

Maya runs `npx 3c dashboard` in her terminal, then opens `.3c/dashboard.html` in her browser. Three panels: which gates fired most today, what proportion of her commits passed the gate on first try, and the trend of post-edit findings across the sprint. **Her telemetry is local-only. Gitignored. Never egressed at L2. Her manager cannot see it.** A delivery view beside the panels reads the repository's handover issues through gh from GitHub, cached locally; it uploads nothing.



What 3C does for you: rolls up the day's local JSONL events into three signals you can act on tomorrow. **What Maya still owns:** what to do about a 78% pass rate — push it higher with practice, or sharpen the rule file itself so the next refusal explains itself properly? **What 3C does NOT do at L2:** share this with anyone. Not the team. Not the manager. The L3 team-observability dashboard is a different surface, opt-in per team, on a self-hostable backend — not a stealth pipe from this view.

KEY TAKEAWAY

> A dashboard your manager cannot see is the only honest version of personal observability.

COMMON MISTAKES · PINNABLE CHECKLIST

The assistant types. You instruct.

Six wrong turns most teams make in their first week with 3C, and the one rule that summarises the rest. Print this page, pin it next to your laptop — week one is where these wrong turns cost the most.

01 · Rubber-stamping AI issues

`/3c-create-issue` produces what you prompt, not what you mean. **Read every issue body before publishing.**

02 · Pasting the standards doc

Trust `.claude/rules/` and the 150-instruction budget. Re-pasting blows the budget and degrades the model.

03 · Bypass without a real reason

The JSONL event is permanent and reviewable. **"forgot why"** in the reason field is how teams lose trust in the bypass mechanism.

04 · Treating boundary findings as noise

A post-edit finding is the architecture *your team declared*, not a style opinion. **Fix the direction** — or change `.3c/architecture.yaml` on purpose.

05 · Implement on un-enriched issues

Enrich first (Phase 2), then implement. `architect` is cheaper before code than after.

06 · Hand-editing to "save time"

The assistant types, you instruct. Every hand-edit is a missed opportunity for the rule library to evolve.

Pinnable checklist · pin next to your laptop

Slash commands by phase: `/3c-create-issue` · `/3c-project-manager` · `/3c-implement` · `/3c-test` · `/3c-review` before the PR

Terminal commands (run in your shell, not as slash commands): `npx 3c bug --summary "…" --repro "…" --expected "…" --actual "…"` · `npx 3c bypass --reason="..."` · `npx 3c dashboard` (after `npm install` + `npx 3c setup`)

Subagents — when to dispatch: `architect` for design calls · `security-reviewer` for auth/crypto/input-handling · `code-reviewer` pre-PR · `test-author` to draft the failing test

that `/3c-test` writes and runs

Gate chain (firing order): Pre-Commit (<10s) → Security Scan (<60s); the post-edit check (~300ms) is advisory and blocks nothing

Bypass syntax (terminal): `npx 3c bypass --reason="<one-line>"` — logged, never silent

File paths to remember: `.claude/CLAUDE.md` (≤150 instructions) · `.claude/rules/<rule>.md` · `.3c/architecture.yaml` · `.3c/telemetry/hooks/*.jsonl` · `.3c/dashboard.html`

The one rule that summarises the rest: the assistant types, you instruct.

3C — Claude Code Coach

Open source · MIT licensed · built in public

github.com/kgn-git/3C

#ai-coding · #governance